

# TD7: Ingeniería de Datos

Resumen — Segundo Parcial

Universidad Torcuato Di Tella

---

## 8.1 — Calidad y Gobernanza de Datos

---

¿Qué es la calidad de datos?

**Definición:** la **aptitud de un dato para ser usado** por quien debe consumirlo. Si los datos son malos, habrá consecuencias negativas: decisiones incorrectas, procesos rotos, pérdida de confianza.

La calidad se mide y se cuantifica. No es una opinión: hay métricas concretas.

¿Cómo se mide?

- **Análisis univariado:** valor mínimo/máximo, media, mediana, moda, tablas de frecuencia, gráficos (histogramas, boxplots). Permite detectar outliers y distribuciones inesperadas.
- **Análisis bivariado:** coeficiente de correlación, tablas de contingencia, scatter plots (diagramas de dispersión). Permite detectar relaciones entre variables.

### Las 8 dimensiones de calidad de datos

Estas son las 8 formas en las que un dato puede ser “malo”. Cada una mide un aspecto diferente:

1. **Precisión:** ¿qué tan cercano a la realidad es el dato? Ejemplo: un apellido “Perez” tiene formato correcto (es un string), pero *no se corresponde con la realidad observable* — el apellido real es “Perez”. El dato es impreciso.
2. **Compleitud:** los datos no deben tener valores desconocidos (NULLs) donde no debería haberlos. Ejemplo: un cliente sin “Total de gastos” registrado es un dato incompleto.
3. **Consistencia:** las reglas de negocio se mantienen entre tablas — los datos tienen coherencia. Ejemplo: si la tabla de empleados dice que Juan trabaja en la empresa con código 281 cuya dirección es “Av. Figueroa Alcorta 7350”, pero la tabla de empresas dice que la empresa 281 tiene dirección “Juan Pablo Saenz Valiente 1021”, hay *inconsistencia*.
4. **Puntualidad:** el dato debe estar disponible *on-demand*, cuando se lo necesita. No sirve tener datos correctos que llegan tarde.
5. **Validez:** revisión de **tipo de dato** y **claves foráneas**. Ejemplo: un campo “Total de gastos” con valor “a” no es válido (no es numérico). Una FK con código de país 99 que no existe en la tabla de países tampoco es válida. Un email “hola@gmail” sin dominio completo tampoco.
6. **Unicidad:** no guardar varias veces el mismo dato. No debe haber duplicados del mismo hecho o entidad.
7. **Conformidad:** el dato debe estar en el formato correcto esperado.
8. **Accesibilidad:** el dato debe estar disponible para los interesados que lo necesitan.

### Gobernanza de datos

**Definición:** la **capacidad de una organización de asegurar que los datos sean de alta calidad**. Se implementan procesos y prácticas para gestionar efectivamente los datos.

La gobernanza abarca:

- Quiénes son los **responsables** de conseguirla (data stewards)

- Cómo los datos van a ser **almacenados, respaldados y protegidos**
- **Normas** sobre cómo serán utilizados
- **Controles y procedimientos** de auditoría

### Ciclo de vida del dato

Todo dato pasa por estas etapas:

**Create → Store → Use → Share → Archive → Destroy**

La gobernanza debe acompañar al dato en *cada una* de estas etapas.

### Alcanzando la calidad

#### Pipelines de datos

Cuando se pasan datos entre dos equipos o subsistemas, estos **se deben validar** para que lleguen en el estado esperado. Si el pasaje se hace en streaming, hay que validar continuamente el flujo, definiendo *antes* qué hacer si alguna regla se rompe.

#### Observabilidad

**Definición:** la comprensión que tiene una organización respecto a la **salud y desempeño de los datos** en sus sistemas. A través de la observabilidad se alcanza y se mejora la calidad.

Se logra mediante estas técnicas:

- **Monitoreo automatizado:** alertas cuando algo se desvía
- **Análisis de causa raíz (RCA):** ir al origen del problema, no solo al síntoma
- **Data lineage:** rastreo de cómo se llega a un dato (de dónde viene, qué transformaciones sufrió)

Estas técnicas se usan para **prevenir, detectar y resolver anomalías** lo antes posible.

**Recordá:** entre más se tarda en detectar una anomalía, peor es la consecuencia. Un error detectado en la carga es barato; un error detectado después de que un dashboard ejecutivo tomó decisiones basadas en datos erróneos es carísimo.

## 8.2 — Privacidad y Compliance

### Roles clave

- **Data Governance Lead:** define la **estrategia** de organización en cuanto a la gobernanza de los datos. Es el rol estratégico.
- **Data Steward:** quien se encarga de ejecutar las **tareas diarias** de gobernanza. Es el rol operativo.

**Recordá la diferencia:** el Lead define *qué* hacer y *por qué*. El Steward ejecuta el *cómo*, día a día.

### PII (Personally Identifiable Information)

**Definición:** información que permitiría **identificar en forma directa a un individuo**. Ejemplos: nombre, dirección, CUIT, teléfono, e-mail.

Las organizaciones deben cumplir normativas de protección (GDPR, CCPA, Ley 25.326). La gobernanza de datos implica **rastrear dónde vive y fluye PII** en todos los pipelines, y controlar quién tiene acceso.

## Datos sensibles prohibidos (Argentina)

La ley argentina establece que **no puede haber** registros referidos a: origen racial o étnico, opiniones políticas, convicciones religiosas, filosóficas o morales, afiliación sindical, o datos relacionados con la salud o la vida sexual.

### Legislación

- **Argentina — Ley 25.326/2000:** los ciudadanos pueden solicitar que sus datos no sean usados sin consentimiento, que les informen qué datos personales están registrados, que sean corregidos/actualizados/suprimidos, y que sean guardados confidencialmente. Pueden iniciar acción judicial. Organismo de aplicación: **AAIP** (Agencia de Acceso a la Información Pública).
- **Unión Europea — GDPR (2016):** Reglamento General de Protección de Datos.
- **EEUU — CCPA (2018):** California Consumer Privacy Act.

**Recordá:** las filtraciones de datos son costosas y dolorosas, al igual que las multas regulatorias. La protección de datos no es solo ética, es una obligación legal.

## Data Catalogs

**Definición:** metadatos curados que reflejan **qué datos** tiene una organización, **quién** los usa y **dónde** se los guarda.

Los data catalogs funcionan como un **inventario de metadatos** y dan a los interesados la información necesaria para evaluar la accesibilidad, la salud y la ubicación de los datos. También pueden integrarse con machine learning y automatización para hacer los datos más descubribles.

Sirven para: gobernanza y compliance, rastrear dónde vive y fluye PII, saber quién tiene acceso a qué datos, responder “¿dónde busco mis datos? ¿qué representan? ¿son relevantes?”.

### Tipos de catálogos:

- **In-house:** contruidos internamente por la empresa. Ejemplos: Uber (Databook), Airbnb, Netflix.
- **Third-party:** herramientas comerciales o de proveedores cloud (Google, AWS, Azure).
- **Open Source:** Amundsen (Lyft, 2018), Apache Atlas, Magda, CKAN.

## Data Discovery

**Definición:** el proceso de extraer insights y patrones significativos de grandes volúmenes de datos. No se trata de recolectar más datos, sino de **darle sentido a los datos que ya se tienen**.

Data Discovery reemplaza la necesidad de un catálogo estático al proveer un entendimiento **dinámico y en tiempo real** del estado actual de los datos (no su estado “ideal catalogado”). Permite la **democratización del dato**: que todos puedan acceder a los datos que necesitan para su trabajo.

### Niveles de madurez en una organización

1. **Data reactive:** nadie accede ni usa datos en su trabajo diario. La empresa rara vez reporta o presenta datos.
2. **Data scaling:** muy pocas personas tienen las habilidades y el acceso necesario para analizar, reportar y presentar datos con confianza.
3. **Data progressive:** cada equipo tiene al menos **un empleado fluido** en datos que puede analizar, reportar y presentar, sin importar su rol.

4. **Data fluent: todos** saben cómo acceder a los datos que necesitan para hacer su trabajo (esto no significa que todos necesiten saber programar).

## Data Literacy y Swampification

**Data literacy** = la habilidad de leer, escribir y comunicar sobre datos de una manera que genere valor e impacto para la organización. Si no se entiende el valor de los datos, no se puede entender el valor de la calidad de datos.

**Swampification** ocurre cuando se depende demasiado de un data engineer o data scientist para saber dónde residen ciertos datos, quiénes son sus stakeholders y cómo podrían cambiar. Los datos se “pantanizan”: la curva de aprendizaje para entender los datos es tan empinada que nadie puede trabajar con ellos.

## 8.3 — Optimización en SQL Server

### Planes de ejecución

SQL Server genera un **plan físico** visible en el query optimizer. Es la estrategia concreta que el motor elige para ejecutar una consulta.

**Recordá:** el flujo de datos va de **derecha a izquierda** en el diagrama. El porcentaje en cada operador indica su **costo relativo** dentro del plan total. Un operador con 95 % es el cuello de botella.

### Operadores de acceso a datos

Hay dos formas fundamentales de acceder a datos: **Scan** (recorrer todo) y **Seek** (ir directo por índice). Según la estructura de la tabla:

| Estructura                  | Scan                 | Seek                 |
|-----------------------------|----------------------|----------------------|
| Heap (sin índice clustered) | Table Scan           | — (no es posible)    |
| Clustered Index             | Clustered Index Scan | Clustered Index Seek |
| Nonclustered Index          | Index Scan           | Index Seek           |

**Seek** = acceso directo por índice, va a las filas que cumplen la condición. Eficiente.

**Scan** = recorre toda la estructura secuencialmente. Costoso en tablas grandes.

**Recordá:** una Heap **nunca puede hacer Seek** porque no tiene índice clustered — sus datos no tienen orden. Solo puede hacer Table Scan (recorrer todo).

### Operadores de agregación

SQL Server usa dos operadores para funciones como SUM, AVG, MAX, GROUP BY, DISTINCT:

#### Stream Aggregate

- Para **scalar aggregates** (sin GROUP BY, ej: SELECT COUNT(\*) FROM tabla) → **siempre** usa Stream Aggregate.
- Con GROUP BY → los datos **deben venir ordenados** por la columna de agrupación. Si hay un índice que provea ese orden, lo usa directo. Si no, el optimizador inserta un operador **Sort** antes.

## Hash Aggregate (Hash Match)

- Para tablas grandes con datos **no ordenados**
- **No necesita que los datos estén ordenados:** construye una hash table en memoria
- Óptimo cuando la cardinalidad estimada (cantidad de grupos) es baja

¿Cuándo elige uno u otro?

Si se crea un índice que provee datos ordenados, el optimizador puede elegir Stream en vez de Hash (más eficiente). Si la entrada no está ordenada pero se pide orden explícito, el optimizador puede: (a) insertar Sort + Stream Aggregate, o (b) usar Hash Aggregate + Sort al final.

## DISTINCT

DISTINCT puede implementarse con Stream Aggregate, Hash Aggregate o un operador **Distinct Sort**.

## Otros operadores

- **Compute Scalar:** realiza cálculos o conversiones de tipo sobre los datos. Ejemplo: COUNT(\*) retorna **int** pero COUNT\_BIG(\*) retorna **bigint**; Compute Scalar convierte el tipo.
- **Filter:** aplica una condición de filtrado **posterior** a la agregación, típicamente para cláusulas HAVING. Se ejecuta después del aggregate.

## Juntas (Joins)

El optimizador de consultas debe tomar **dos decisiones importantes** con respecto a las juntas:

1. La selección de un **orden de las juntas** (qué tabla va como outer, cuál como inner)
2. La selección del **algoritmo de junta** (Nested Loops, Merge o Hash)

**Recordá:** Merge Join y Hash Join **requieren** que la junta tenga al menos un predicado basado sobre un **operador de igualdad** (equi-join, ej: ON a.id = b.id). Nested Loops es el único que acepta cualquier predicado.

## Nested Loops Join

- Es muy efectivo si el **outer input es suficientemente pequeño** y el **inner input es grande pero indexado**
- Por cada fila del outer, busca en el inner (de ahí el nombre “loops”)
- Es el **único** que acepta cualquier predicado de junta (no solo equi-join)
- Inner table: **debe tener índice**. Outer table: preferible que sea pequeña

## Merge Join

- Requiere que **ambas tablas estén ordenadas** por la columna de join
- Si ya hay un clustered index o covering index que provea ese orden, no necesita Sort previo. Si no, SQL Server introduce un operador Sort explícito
- Óptimo para tablas **grandes** con clustered o covering index en ambas
- Solo soporta **equi-join**

## Hash Join

- Inner table: **no necesita índice**. Outer table: índice opcional
- Útil para tablas de **cualquier tamaño**
- **No requiere** datos preordenados: construye una hash table con la tabla más chica y luego busca matches
- Solo soporta **equi-join**
- Condición óptima: outer table pequeña, inner table grande
- Implementado con el operador Hash Match

## Tabla resumen de Joins

| Algoritmo    | Inner table       | Outer table | Tamaño         | Presorted | Cláusula   |
|--------------|-------------------|-------------|----------------|-----------|------------|
| Hash Join    | Sin índice        | Opcional    | Cualquiera     | No        | Equi-join  |
| Merge Join   | Con índice        | Con índice  | Grande         | <b>Sí</b> | Equi-join  |
| Nested Loops | Con índice (must) | Preferable  | <b>Pequeño</b> | Opcional  | Cualquiera |

## 9 — Seguridad de Bases de Datos

### ¿Por qué seguridad?

La seguridad de la información es un área amplia que aborda tres grandes problemas:

1. Cuestiones **legales y éticas** relacionadas con el derecho a acceder a cierta información. Cierta información puede considerarse privada y no pueden acceder legalmente organizaciones o personas no autorizadas.
2. Cuestiones de **política a nivel gubernamental, institucional o corporativo** con respecto a la información que no debería ponerse a disposición de todos. Ejemplo: calificaciones crediticias, registros médicos personales.
3. Necesidad de **identificar múltiples niveles de seguridad** y categorizar los datos y los usuarios basándose en estas clasificaciones. Niveles: alto secreto, secreto, confidencial y no clasificado.

### Tríada CIA

La gobernanza, privacidad y compliance se nutren a partir de la capacidad de una empresa de mantener segura la información. Esto implica garantizar tres propiedades:

- **Confidencialidad:** la información solo la pueden ver quienes **estén autorizados** a verla.
- **Integridad:** **no pueden haber modificaciones** de la información por parte de personas no autorizadas.
- **Disponibilidad:** la información está **disponible** para los que estén autorizados **en el momento que la necesiten**.

### Estrategias y técnicas de seguridad

- **Autenticación:** verificación de identidad (¿sos quien decís ser?)
- **Control de acceso:** verificación de que los actores accedan a la información correcta (¿tenés permiso para ver esto?)
- **Encriptación:** oscurecimiento de la información si cae en manos equivocadas

- **Firewalls o sistemas de detección:** detección y prevención del acceso a redes y sistemas

## Medidas de control en bases de datos

### 1. Control de inferencia

Evitar que usuarios deduzcan información confidencial a partir de datos a los que *sí* tienen acceso.

**Ejemplo clásico:** si tuviéramos una base de datos para estadísticas de población y un usuario puede consultar el promedio de sueldos de un departamento que tiene **una sola persona**, infiere el sueldo exacto de esa persona. Un usuario no debería poder acceder a información confidencial sobre alguien en específico, ni debería poder inferir ciertas cosas a partir de las estadísticas.

### 2. Cifrado de datos (Data Encryption)

Sirve para proteger datos confidenciales (contraseñas, tarjetas de crédito) que viajan a través de algún tipo de red. El usuario no autorizado no podrá descifrar el dato. El usuario autorizado recibirá el algoritmo de decodificación para descifrar el mensaje.

**Cifrado simétrico:** misma clave para cifrar y descifrar (más rápido). **Cifrado asimétrico:** par de claves pública/privada (más seguro para intercambio de claves).

### 3. Control de acceso

El **administrador central de la base de datos (DBA)** tiene la responsabilidad de la seguridad general y la autoridad para gestionar el sistema. Esto es: otorgar privilegios a los usuarios que requieran acceso y clasificar usuarios y datos según la política de la organización.

El DBA tiene una cuenta especial en la base de datos que se llama **superusuario**. Este título otorga facultades que no tienen las demás cuentas sin privilegios, como otorgar y revocar privilegios para cuentas individuales o grupos de cuentas.

## Modelos de control de acceso

### DAC (Discretionary Access Control)

El método por preferencia se basa en el **otorgamiento y revocación de privilegios**. Se incluyen declaraciones en el lenguaje de consulta (GRANT, REVOKE) que permitan al DBA y a algunos usuarios hacerlo posible. Un usuario tiene o no tiene un privilegio en específico. **No hay grises.**

### MAC (Mandatory Access Control)

Es una **política de seguridad adicional** que clasifica datos y usuarios según **clases de seguridad**. Este método se **combina con DAC**: primero se aplican los privilegios discrecionales (tenés un privilegio o no), y **además** se chequea que el nivel de seguridad del usuario sea compatible con el nivel del dato al que quiere acceder.

### RBAC (Role-Based Access Control)

Los privilegios y otros permisos están **asociados con roles organizacionales, no con usuarios individuales**. Se pueden asignar varios usuarios a un rol. Un **rol** agrupa funciones y niveles de responsabilidad. Implican una **jerarquía** donde cada rol hereda las capacidades de sus ancestros.

### Conceptos:

- Un **usuario** es un miembro de la organización con algún acceso a los datos
- Un **objeto** es un elemento sobre el cual se puede controlar el acceso

### Jerarquía de objetos:

1. **Base de datos:** conjunto de esquemas y un único sistema de permisos. Puede haber más de una base en el mismo motor.
2. **Esquema:** separación lógica de objetos dentro de una base de datos. Conjunto de objetos.
3. **Objetos atómicos:** tablas, vistas, funciones, índices...

**Recordá el ejemplo:** una empresa puede tener roles como “cuenta de ventas”, “gerente”, “agente de compras”. En vez de dar permisos individuales a cada persona, se asignan a un rol y los usuarios se asignan al rol. Mucho más simple de administrar.

### SQL Injection

Las bases de datos no están expuestas directamente a redes externas, pero se puede acceder de forma indirecta a través de servicios que *sí* se exponen. Los ataques de **SQL Injection** se aprovechan del hecho de que **algunos servicios y aplicaciones usan input externo como parte de sus consultas** a la base de datos.

Este ataque está orientado a: obtener información no autorizada, alterar información, o destruirla.

**Recordá el ejemplo de Bobby Tables:** el nombre del hijo es Robert'); DROP TABLE Students;-. Al usar ese input directamente en un SQL, se ejecuta un DROP TABLE. Prevención: **sanitizar inputs** y usar prepared statements.

## 10.1 — Password Hashing

### El problema

Las contraseñas son información **ultra sensible** porque pueden permitir identificarse en más de un sistema. Se reciben de parte del usuario cuando crea su cuenta, pero también en cada autenticación posterior. Se debe poder validar en los registros cada vez que se la necesite. Ningún tercero debe poder conseguirlas mediante un ataque. **Ningún usuario interno con privilegios debería poder conseguirlas si quisiera.**

**Solución:** oscurecerlas mediante **hashing**.

### Propiedades de una función hash criptográfica

- **Determinística:** un texto plano devuelve **siempre el mismo hash**
- **No reversible:** a partir de un hash hay múltiples posibles textos planos (no se puede volver atrás)
- Es **computacionalmente inviable** obtener el texto plano a partir de un hash

### ¿Cómo funciona en la práctica?

1. Al **crear la contraseña:** se calcula el hash y se **almacena solo el hash** (nunca el texto plano)
2. En cada **autenticación:** se calcula el hash del input del usuario y se compara con el hash almacenado
3. Ante una **filtración:** incluso si alguien roba la base de datos, solo ve hashes — no puede obtener las contraseñas originales (ni siquiera un superusuario)



## Rainbow Tables

Pero el hashing solo no es suficiente. Existen **rainbow tables**: listas de hashes **precalculados** que sirven para descifrar contraseñas comunes. Estas tablas comparan muchos hashes muy rápido para encontrar matches. No pueden desenscriptar todas las contraseñas, pero sí una gran parte de las más comunes.

## Salting

Es una técnica para **dificultar el ataque por rainbow tables**:

1. Para cada contraseña a almacenar, se genera un **string al azar** (comúnmente llamado **salt**)
2. Se **almacena el salt** junto con la información de la cuenta
3. Cada vez que se tenga que calcular el hash (al crear o validar la contraseña), se **concatena el salt con la contraseña original**: `hash(contraseña + salt)`

**Recordá por qué funciona:** dos usuarios con la *misma* contraseña tendrán **hashes distintos** porque cada uno tiene un salt diferente. Las rainbow tables se vuelven inútiles porque necesitarían precalcular hashes para cada posible combinación de contraseña + salt.

## 10.2 — Big Data: Motivación

### Las 4 V's de Big Data

- **Volumen:** cantidad masiva de datos. Estamos hablando de terabytes, petabytes o más.
- **Variedad:** datos estructurados (tablas), semi-estructurados (JSON, XML) y no estructurados (imágenes, texto libre, logs).
- **Velocidad:** ritmo de generación de datos y necesidad de procesarlos rápidamente. Algunos casos requieren procesamiento en tiempo real.
- **Veracidad:** calidad y confiabilidad de los datos. No todos los datos son igualmente confiables.

### Escalabilidad

**Vertical (scale up):** agregar más recursos (CPU, RAM) a un mismo servidor. Tiene un techo físico y económico.

**Horizontal (scale out):** agregar más servidores/nodos al sistema. Big Data requiere escalabilidad horizontal porque los volúmenes exceden la capacidad de un solo servidor.

## 10.3 — Big Data: Recorrido inicial

### Evolución del sistema (SuperWebAnalytics)

Este ejemplo muestra cómo un sistema simple se va complejizando ante el crecimiento:

1. **Estado inicial:** una RDBMS simple con una tabla de pageviews (`user_id`, `url`, `date`, `pageviews`). Ante cada evento se actualiza el registro correspondiente.
2. **Aumenta el tráfico:** la DB no puede mantener el ritmo de escrituras → timeouts. **Solución:** desacoplar componentes con una **cola (queue)** y un **worker** independiente que acumula eventos para reducir llamadas a la base. Esto maneja la **back-pressure** (el productor no se frena por el consumidor).
3. **Aumenta aún más el tráfico:** el worker se atrasa y la cola solo crece. **Solución:** paralelizar agregando **más workers** con un **load balancer** que distribuye la carga equitativamente.

4. **La base de datos es cuello de botella:** se aplica **sharding** (partición horizontal): desparramar los datos en **múltiples servidores de DB**. Cada servidor tiene un subconjunto de filas. La asignación se hace con distribución uniforme. Las queries requieren consultar todos los shards y hacer merge de resultados. Si la app crece más, hay que hacer **resharding** sin dejar de recibir tráfico.
5. **Fallas en nodos:** más nodos = más frecuencia de fallas de disco. Se necesita hacer la DB **fault tolerant** → registrar qué nodos están online y mantener **réplicas pasivas**.
6. **Fallas en el código (bugs):** un bug en los workers puede **corromper la información** (source of truth = DB). Los backups no alcanzan porque implican perder información y no se puede saber qué está corrupto. Se necesita **human fault tolerance**.

**Recordá la conclusión:** el sistema se fue complejizando paso a paso, y cada solución trajo nuevos problemas. Big Data nace de esta necesidad de manejar volumen, velocidad, fallos y complejidad creciente.

## First Principles

El **objetivo** de un sistema de datos es **contestar preguntas basándose en información recolectada**.

No toda la información es igual: cierta información es derivada de otra. Si se busca el origen, se llega a una fuente que no es derivable de ninguna otra.

**Data** = información que es verdadera simplemente por existir. Es la fuente desde la cual **toda otra información puede ser derivada**. Son los “axiomas” del sistema.

La definición más general de un sistema de datos:

$$\text{query} = \text{function}(\text{all data})$$

**Requerimientos no funcionales deseables:** (i) tolerancia a fallos, (ii) escalabilidad, (iii) extensibilidad, (iv) mantenibilidad, (v) baja latencia, y (vi) soporte para consultas específicas.

## Data Warehouse vs Data Lake

Ambos son **arquitecturas con fines analíticos y de reporting**. Son no transaccionales (baja o nula mutabilidad), repositorios centrales que unifican muchas fuentes diversas, orientados a grandes volúmenes y pensados para escalabilidad.

### Data Warehouse (schema-on-write)

Datos fuertemente estructurados. Las estructuras se definen **antes** de la carga y los datos se validan contra ellas. La estructura se orienta a los casos de análisis y reporting necesarios, impuesta mediante ETL.

**Ventajas:** las lecturas son más rápidas (la penalización está en la carga), la relación entre entidades es sencilla, los datos quedan documentados en la estructura.

**Desventajas:** cualquier cambio en el esquema obliga a reprocesar, es complicado mapear datos no estructurados, es costoso mantener el nivel de detalle más atómico para volúmenes altos.

**Componentes principales:** almacenamiento (RDBMS en distintas capas), pipelines de ETL, capa de metadata, data marts (capa de consulta para casos específicos), y capa de presentación (análisis y reporting).

### Data Lake (schema-on-read)

Datos crudos de las distintas fuentes con estrategia **schema-on-read**: no se impone estructura al ingestar (ELT), no se descartan ni limpian datos. Acepta datos estructurados y no estructurados. Los datos son

inmutables.

**Ventajas:** los volcados son más rápidos, hay flexibilidad para leer según la vista que interese en cada consulta, es apropiado para datos no estructurados, permite guardar el nivel de detalle más atómico.

**Desventajas:** la penalización está en la lectura, no hay documentación clara de los datos, necesita alta capacidad de proceso, tiene más dificultades para lograr data governance y seguridad.

### Pipelines de ingestión: ETL vs ELT

**ETL (Extract, Transform, Load):** proceso por el cual la información fluye entre 2 sistemas en 3 etapas. (1) **Extract:** se consulta y extrae del sistema origen. (2) **Transform:** se limpia, normaliza y procesa para adaptarse al destino. (3) **Load:** se almacena *ya transformada* en el destino. ETL no es propio de ninguna arquitectura particular.

**ELT (Extract, Load, Transform):** mismo objetivo pero con **orden distinto**. (1) **Extract:** se consulta y extrae del sistema origen. (2) **Load:** se almacena *cruda* en el destino. (3) **Transform:** se limpia, normaliza y procesa **en el destino**. Asociado a Data Lakes y schema-on-read.

**Recordá la diferencia clave:** en ETL la transformación ocurre *antes* de cargar (los datos llegan limpios). En ELT la transformación ocurre *después* de cargar (los datos llegan crudos y se procesan en destino).

## 10.4 — Integración de Datos

### OLTP vs OLAP

**OLTP (Online Transaction Processing):** optimizada para operaciones transaccionales (INSERT, UPDATE, DELETE) de muchos usuarios concurrentes. Es la base de datos “del día a día” de la aplicación.

**OLAP (Online Analytical Processing) / DWH:** optimizada para consultas analíticas complejas sobre grandes volúmenes de datos históricos. Es donde se hacen los reportes y análisis.

**Recordá:** OLTP = muchas escrituras pequeñas y rápidas. OLAP = pocas lecturas grandes y complejas (queries que agregan millones de filas).

### Arquitectura Medallion

Patrón de organización de datos en un Data Lake con 3 capas progresivas de refinamiento:

- **Bronze:** datos crudos sin procesar, tal cual llegan de las fuentes. Es el “landing zone”.
- **Silver:** datos limpiados, conformados, deduplicados. Se aplican reglas de calidad.
- **Gold:** datos agregados, modelados y listos para consumo analítico/BI. Es lo que ven los usuarios finales.

### Star Schema (Esquema Estrella)

Modelo de datos optimizado para DWH. Tiene una forma de estrella con una tabla central rodeada de tablas satélite:

- **Fact table (tabla de hechos):** tabla central que contiene las **métricas/medidas numéricas** del negocio (ej: monto de venta, cantidad vendida, duración) y las **foreign keys a las dimensiones**. Suele tener **muchas filas** (cada fila es un evento/transacción).

- **Dimension tables (tablas de dimensiones):** tablas descriptivas que proveen **contexto** a las medidas de la fact table. Contienen atributos como nombre de producto, categoría, fecha, nombre de cliente. Suelen tener **pocas filas pero muchas columnas**.

**Ejemplo:** la fact table de “Ventas” tiene columnas: `fecha_id`, `producto_id`, `cliente_id`, `monto`, `cantidad`. Las dimension tables son: `dim_fecha` (año, mes, día, trimestre), `dim_producto` (nombre, categoría, precio), `dim_cliente` (nombre, ciudad, segmento).

## Slowly Changing Dimensions (SCD)

¿Qué pasa cuando los datos de una dimensión cambian? (ej: un cliente se muda de ciudad)

- **Tipo 1:** se **sobreescribe** el valor anterior. Simple pero se **pierde el historial**. No se puede saber cuál era el valor antes del cambio.
- **Tipo 2:** se **agrega una nueva fila** con fechas de vigencia (`valid_from`, `valid_to`), conservando el **historial completo**. La fila anterior se marca como expirada. Más complejo pero permite análisis histórico.

## Orquestación

**DAG (Directed Acyclic Graph):** grafo dirigido sin ciclos que define las **dependencias entre tareas** de un pipeline de datos. Cada nodo es una tarea y las aristas indican precedencia (“esta tarea debe completarse antes de que empiece esta otra”).

### Conceptos clave de orquestación:

- **Idempotencia:** una operación es idempotente si ejecutarla **múltiples veces produce el mismo resultado** que ejecutarla una vez. Es fundamental para poder reintentar tareas fallidas sin generar datos duplicados o corruptos.
- **Backfill:** proceso de **ejecutar retroactivamente** un pipeline para fechas/períodos pasados. Se usa cuando se agrega una nueva transformación o se corrige un bug y hay que reprocesar datos históricos.
- **Stack moderno visto en la materia:** PostgreSQL (base de datos) + dbt-core (transformación) + Dagster (orquestación) + Metabase (visualización/BI).

## 10.5 — Arquitecturas de Datos

### Fully Incremental vs Event Sourcing

Son dos enfoques opuestos para manejar el estado de un sistema:

**Fully Incremental:** cada nuevo dato actualiza incrementalmente el estado actual. Se mantiene **solo el estado más reciente**. Es lo que hace una base relacional tradicional con UPDATEs. **Problema:** si hay un bug que corrompe datos, no se puede reconstruir el estado porque los datos originales se perdieron (fueron sobrescritos).

**Event Sourcing:** almacena **todos los eventos/cambios** como una secuencia inmutable (nunca se borran). El estado actual se deriva **recomputando todos los eventos** desde el inicio. **Ventaja:** historial completo y capacidad de recomputar desde cero. **Desventaja:** la latencia del recompute puede ser alta si hay muchos eventos.

**Recordá el ejemplo de la red social de Tom:** en fully incremental, se guarda “Tom tiene 3 amigos” (estado actual). En event sourcing, se guarda la secuencia de eventos: “17/may Add Alice, 18/may Add Jerry, 19/may Add Charlie, 20/may Remove Jerry, ...”. A partir de la secuencia se

puede derivar tanto la lista actual de amigos como la cantidad — y cualquier otra pregunta futura.

## Modelos de consistencia

En sistemas distribuidos (datos replicados en varios nodos), ¿qué garantías tenemos sobre la frescura de los datos que leemos?

- **Consistencia fuerte (strong consistency):** después de una escritura, **toda** lectura subsiguiente (desde cualquier nodo) devuelve el valor actualizado. Todos los nodos ven los mismos datos al mismo tiempo. Es la garantía más fuerte pero la más costosa.
- **Consistencia eventual (eventual consistency):** si no se hacen más escrituras, eventualmente **todas las réplicas** convergerán al mismo valor. No garantiza que una lectura inmediata después de una escritura devuelva lo más reciente. Es la más débil pero la más performante.
- **RYOW (Read Your Own Writes):** garantiza que un usuario siempre ve **sus propias escrituras**. Otro usuario podría ver datos desactualizados, pero cada uno ve al menos lo que él mismo escribió.
- **Consistencia de sesión:** garantiza consistencia RYOW dentro de una **misma sesión** de usuario. Entre sesiones diferentes puede haber inconsistencia.

## Teorema CAP

En un sistema distribuido, **solo se pueden garantizar 2 de 3** propiedades simultáneamente:

- **Consistency:** todos los nodos ven los mismos datos al mismo tiempo
- **Availability:** toda request recibe una respuesta (no necesariamente con el dato más reciente)
- **Partition tolerance:** el sistema sigue funcionando pese a fallos de comunicación entre nodos

**Recordá la implicancia práctica:** las particiones de red son **inevitables** en sistemas distribuidos (P es obligatorio). Entonces la elección real es: ante una partición de red, ¿priorizamos **consistencia** (rechazando requests hasta que los nodos se sincronicen) o **disponibilidad** (respondiendo con datos que podrían estar desactualizados)?

## Las 8 Falacias de la Computación Distribuida

Son suposiciones erróneas que los programadores novatos hacen al diseñar sistemas distribuidos:

1. La red es confiable
2. La latencia es cero
3. El ancho de banda es infinito
4. La red es segura
5. La topología no cambia
6. Hay un solo administrador
7. El costo de transporte es cero
8. La red es homogénea

Todas son **falsas**. Un buen sistema distribuido debe diseñarse asumiendo que **todas estas cosas van a fallar**.

## 11.1 — Anonimización y Oscurecimiento de Datos

### ¿Por qué anonimizar?

La anonimización de datos sensibles busca proteger los derechos y la privacidad de las personas, así como preservar la integridad ética de los procesos. La exclusión de los datos garantiza que cierta información personal identificable no sea revelada, lo cual es esencial para dar cumplimiento a la **Ley de Protección de Datos Personales (Ley 25.326)**.

### El proceso de anonimizar

1. **Evaluación previa:** identificar información sensible en los conjuntos de datos
2. **Aplicar técnicas:** elegir y aplicar las técnicas de anonimización adecuadas
3. **Gestionar riesgos:** gestionar los riesgos de identificación y divulgación
4. **Documentar:** registrar claramente los detalles del proceso, los parámetros utilizados y los controles

**Recordá:** la documentación del proceso debe conservarse de forma **segura**, ya que la revelación de los parámetros puede facilitar la **reidentificación** de los datos anonimizados. Paradójicamente, el documento que explica cómo se anonimizó es en sí mismo un dato sensible.

### Técnicas de anonimización

#### Perturbación

Alterar **ligeramente** los valores de los datos. Por ejemplo, añadir ruido aleatorio a los ingresos o edades. Los datos individuales dejan de ser exactos pero las propiedades estadísticas del conjunto se mantienen.

#### K-anonimato

Asegurar que cada registro **no sea distinguible** de al menos  $k - 1$  otros registros. Se modifican los datos para que cada combinación de atributos cuasi-identificadores (ej: edad, código postal, género) aparezca **al menos  $k$  veces**.

Ejemplo con  $k = 3$ : si solo hay una persona de 28 años en código postal 1428, se generaliza la edad a “25-30” para que haya al menos 3 personas con esa combinación.

#### L-diversidad

Garantizar que en cada grupo k-anónimo haya al menos  $l$  valores “**bien representados**” para los atributos sensibles.

**Recordá por qué L-diversidad es necesaria:** si se usa k-anonimato para sueldos y un grupo k-anónimo tiene 5 personas pero *todas con el mismo sueldo*, se puede inferir el sueldo de cualquier persona del grupo. L-diversidad asegura que haya diversidad significativa de valores sensibles dentro de cada grupo.

### Herramientas

- **Amnesia:** herramienta de código abierto (OpenAIRE) para anonimizar datos tabulares. Utiliza k-anonimato y l-diversidad.
- **ARX (Data Anonymization Tool):** código abierto, soporta k-anonimato, l-diversidad, t-closeness y privacidad diferencial. Interfaz gráfica intuitiva.

- **sdcMicro**: paquete R diseñado para anonimización de microdatos, ideal para datos censales y de encuestas. Ofrece supresión, recodificación, perturbación y generación de microdatos sintéticos.

## 11.2 — Arquitectura Lambda: Introducción

### ¿Por qué Lambda?

Vimos que **fully incremental** pierde datos originales (no se puede recomputar) y **event sourcing** tiene alta latencia de recompute. La **Arquitectura Lambda** resuelve los problemas de ambos para casos de uso no triviales, combinando procesamiento **batch** y **real-time** para lograr data freshness con las ventajas del reprocesamiento batch.

### Las 3 capas

- **Batch Layer**: almacena el master dataset y **construye batch views** procesando *todos* los datos. Alta latencia pero resultados completos y precisos.
- **Serving Layer**: sirve e **indexa las batch views** para que las consultas puedan accederlas rápidamente.
- **Speed Layer**: **construye real-time views** con los datos nuevos que llegaron *después* de que empezó el último batch. Compensa la latencia del batch.

### Las 3 ecuaciones que resumen Lambda

Toda la arquitectura se resume en estas tres ecuaciones:

~~query = function(all data)~~ (demasiado caro en tiempo real)

batch view = function(all data)

realtime view = function(realtime view, new data)

query = function(batch view, realtime view)

**Recordá la intuición:** la función `query = function(all data)` podría tomar una gran cantidad de recursos y ser muy cara de computar en tiempo real. La alternativa es **pre-computarla** (batch view). Pero crear una batch view es una operación de alta latencia — para cuando termine, mucha data nueva no estará representada. Para eso se crea una **realtime view** que incorpora la data que quedó afuera del batch. La query final combina ambas vistas.

### Ejemplo conceptual: app tipo Spotify

Aplicación mobile de consumo de contenido. Eventos relevantes generados por usuarios (plays, pausas, favoritos, agregado a listas, etc.) y de otras fuentes (nuevo contenido, ventas físicas, redes sociales). Caso de uso: **sistema de recomendación para usuarios**.

Preguntas clave: ¿qué forma tienen los eventos? ¿dónde los almacenamos? ¿cómo usarlos para entender gustos? ¿cómo hacer que eventos antiguos tengan menos peso?

### Tecnologías asociadas

La Arquitectura Lambda requiere:

- Entornos de cómputo paralelizable (batching y streaming)
- Sistemas de almacenamiento distribuido (filesystems distribuidos, bases NoSQL)



- Frameworks de serialización de objetos
- Sistemas de colas distribuidas (asincrónicas)

### 11.3 — Lambda: Master Dataset

#### ¿Qué es el Master Dataset?

Es el **core de la arquitectura lambda**. Es la **source of truth** (fuente de verdad) de todo el sistema.

**Recordá esta propiedad fundamental:** si se perdieran **todos** los datasets de la serving layer y de la speed layer, se podría **recuperar todo** desde el master dataset. Esto porque las batch views son producidas como función del master dataset, y la speed layer se basa en la data más reciente — todo se podría reconstruir en pocas horas.

Por lo tanto, es la **única parte** de la Arquitectura Lambda que debe estar **absolutamente a resguardo**. Recordar que: máquinas sobrecargadas, discos que fallan, cortes de energía, errores humanos son riesgos intrínsecos que inevitablemente ocurrirán.

#### Datos vs Información vs Consultas vs Vistas

La idea de **dependencia de la información** nos lleva a estas definiciones:

- **Datos (data):** “axiomas” de información que **no pueden ser derivados** de otros elementos. Son la fuente primaria.
- **Información:** el conjunto total de conocimiento (incluyendo los datos) **relevante** al sistema.
- **Consultas (queries):** preguntas que se le pueden hacer a la información con un dominio en mente.
- **Vistas (views):** información **derivada** de los datos base, generalmente orientada a consultas.

**Ejemplo de la red social:** los **datos** son los eventos de usuarios (“17/may Add Alice”, “20/may Remove Jerry”...). La **lista actual de amigos** es una **vista** (derivada de los datos). “¿Tom es amigo de Jerry?” es una **consulta**. La cantidad de amigos actual (3) es otra vista.

#### Las 3 propiedades clave de los datos

##### 1. Crudeza (rawness)

Los datos deben ser **tan crudos (no procesados) como se pueda**. ¿Por qué? Porque cuanto más crudos son, mayor es la capacidad de **derivar más información** en el futuro — incluso información que todavía no se sabe que se busca.

Ejemplo con datos de Spotify: “Ana hizo fav a 3 canciones” es menos crudo que “Ana hizo fav a #829, #287, #7110” que es menos crudo que “Ana hizo fav a #829 el 04/10/2023 a las 03:42 pm en la web”. Cuanto más detalle, más preguntas se pueden responder en el futuro.

Los datos crudos suelen ser no estructurados antes que normalizados. Hay una distinción fina entre simplemente *parsear* los datos y hacer una **normalización semántica**. La normalización semántica (reorganizar información con formato libre en una forma estructurada) implica **eliminar cualidades del dato que pueden importar en el futuro**.

**Recordá:** una arquitectura más cercana a la del **Data Lake** es más consistente con la idea de crudeza. Los algoritmos y formas de procesar los datos pueden cambiar a lo largo del tiempo sin que esto sea un problema — los datos originales siguen ahí.



## 2. Inmutabilidad

Es un concepto extraño pensado desde una base relacional donde el `UPDATE` es una de las operaciones fundamentales. En el master dataset: **no hay updates ni deletes, solo se agrega data** (append-only). Puede haber pocos casos especiales donde se borra información (ej: derecho al olvido).

### Beneficios de la inmutabilidad:

- Hace al sistema **tolerante a las fallas humanas**: si ocurre un error, se puede recomputar el master dataset.
- **Simplifica la carga de datos**: estructuras append-only con archivos planos.

## 3. Perpetuidad

Los datos se **conservan indefinidamente**. No se descartan datos viejos. Comprender los fundamentos de Big Data es asimilar estos conceptos de crudeza, inmutabilidad y perpetuidad.

**Recordá el contraste:** en un ambiente relacional tradicional constantemente se actualiza y resume información para reflejar el estado actual. Los términos “inmutabilidad” y “perpetuidad” no tienen sentido en ese contexto. En Big Data, son fundamentales.

## 11.4 — Almacenamiento del Master Dataset

### El problema

Sabemos que queremos almacenar nuestros datos en archivos por sus ventajas inherentes (append-only, inmutabilidad). Pero los file systems tradicionales no garantizan escalabilidad ni capacidad de procesamiento en paralelo. Necesitamos: (1) un **filesystem distribuido** como infraestructura de almacenamiento, y (2) **frameworks de serialización** para definir cómo se estructuran los datos dentro de los archivos.

### Serialization Frameworks

Los serialization frameworks proporcionan herramientas y bibliotecas para usar objetos de distintos lenguajes. Pueden serializar un objeto en un **array de bytes desde cualquier lenguaje**, y luego deserializar ese objeto en cualquier otro lenguaje. Proporcionan un lenguaje de definición de esquemas para definir objetos y sus campos, y mecanismos para **versionar objetos** de forma segura (un esquema puede evolucionar sin invalidar las versiones previas).

Los tres serialization frameworks notables: **Thrift** (Facebook), **Protocol Buffers** (Google) y **Avro** (Apache).

### Formato Avro (row-oriented)

- Permite almacenar el **esquema junto con el mensaje** (el esquema se define en JSON)
- Aporta **compresión** de los datos
- Organiza la información en **filas (row-oriented)**: optimizado para leer **filas completas** y leer muchas **filas consecutivas**
- Tiene soporte para **estructuras complejas** (registros anidados, arrays, maps, etc.)
- Provee **bibliotecas** en los principales lenguajes para serializar y deserializar objetos
- Organiza los registros en **bloques de tamaño fijo** (muy importante para usar en un filesystem distribuido)

### Estructura interna de un archivo Avro:

- **Header:** un magic number de 4 bytes para identificación, el esquema de serialización (en JSON), metadata extra, y un marcador de sincronización de 16 bytes para indicar separaciones entre bloques.
- **Bloques de datos:** cada uno con un indicador de cuántos registros contiene, indicador del tamaño en bytes total, los objetos serializados, y el marcador de sincronización al final.

### Formato Parquet (column-oriented)

- Organización **columnar**: la organización columnar permite mucha mayor eficiencia de compresión. Establece **encoding específico** para cada columna.
- También permite **particionar** los archivos (para consumo distribuido)
- Brinda optimización para **procesamiento**: solo se leen las columnas que se necesitan (en general, no todas)
- Soporta **predicate pushdown**: acercar la lógica de la recuperación de datos al origen de los datos, ejecutar determinadas operaciones directamente en el origen, para ahorrar tiempo y potenciar el procesamiento al no llevar todos los datos de la red al motor.
- Permite almacenar el **esquema** junto con el mensaje (no tiene lenguaje propio de definición de esquema, se pueden usar los de Thrift o Avro)
- Soporta tipos complejos y cuenta con bibliotecas para los principales lenguajes

### Estructura interna de un archivo Parquet (4 partes):

1. Un **magic number** de 4 bytes para identificación
2. Metadata sobre los **row groups** (los registros están agrupados) incluyendo su offset dentro del archivo
3. Los **column chunks** (primero todos los del row group 1, después los del 2, etc.)
4. El **footer** del archivo, incluyendo el esquema, información de row groups, e información de encoding y ubicación de los distintos chunks

### ¿Cuándo usar Avro vs Parquet?

**Avro (row-oriented):** ideal para **escribir/leer filas completas**, streaming, carga de datos. Cuando se necesitan todos los campos de cada registro.

**Parquet (column-oriented):** ideal para **consultas analíticas** que leen pocas columnas de muchas filas. Mejor compresión. Predicate pushdown. Cuando se quiere hacer un `SELECT columna1, columna2 FROM tabla_enorme`.

### File Systems Distribuidos

Un filesystem distribuido consiste en **múltiples soportes físicos** detrás de un **único file system lógico**. Proveen muchas garantías no funcionales:

- **Escalabilidad:** manejo de grandes volúmenes
- **High availability:** disponibilidad continua
- **Paralelización de lecturas:** múltiples nodos leen en paralelo
- **Redundancia:** datos replicados para tolerar fallos

## Hadoop

**Apache Hadoop** es un framework que permite el procesamiento distribuido de grandes datasets sobre clusters de computadoras usando modelos de programación simples. Está diseñado para escalar desde un servidor hasta miles de máquinas, cada una ofreciendo computación y almacenamiento local.

**Principal ventaja:** diseñado para ejecutar sobre **hardware sencillo, normal y corriente** (commodity hardware). En vez de depender de hardware caro y especializado, la biblioteca misma está diseñada para detectar y manejar fallos a nivel de aplicación.

**Principio clave:** se trata de **acercar el procesamiento al lugar donde se encuentran almacenados los datos** y así aprovechar técnicas de paralelización, aumentando la escalabilidad. Mover el procesamiento es mucho más rápido, fácil y eficiente que mover grandes cantidades de datos, que pueden producir altas latencias y congestión en la red.

**Tres componentes principales de Hadoop:**

- **HDFS (Hadoop Distributed File System):** filesystem diseñado especialmente para Hadoop. Open source, desarrollado en Java, optimizado para archivos muy grandes, manejo de grandes volúmenes en forma distribuida.
- **MapReduce:** modelo de programación para procesamiento distribuido. Divide el trabajo en tareas Map (transformación paralela) y Reduce (agregación de resultados).
- **YARN (Yet Another Resource Negotiator):** sistema de gestión de recursos del cluster. Decide qué tarea se ejecuta en qué nodo y cuántos recursos recibe.

## 12.1 — S3 (Simple Storage Service)

¿Qué es S3?

Amazon S3 es un servicio de **almacenamiento de objetos** en la nube de AWS. No es un filesystem jerárquico tradicional: es un **key-value store** donde cada objeto tiene una clave (key) y un valor (el archivo).

**Recordá:** en la sección de File Systems Distribuidos vimos que un filesystem distribuido presenta **múltiples soportes físicos** detrás de un **único file system lógico**. S3 cumple este rol: abstrae la infraestructura física y brinda escalabilidad, alta disponibilidad y redundancia automáticamente.

**Conceptos clave:**

- **Bucket:** contenedor de objetos. Nombre **globalmente único** en todo AWS. Se asocia a una **región**.
- **Object:** un archivo almacenado dentro de un bucket. Identificado por su **key** (la ruta completa, ej: `datos/ventas/2024.csv`).
- **Region:** ubicación geográfica del data center. Elegir la región más cercana reduce latencia.

S3 no tiene carpetas reales: la apariencia de carpetas es simulada mediante **prefijos** en la key. El archivo `datos/ventas/2024.csv` tiene key `datos/ventas/2024.csv`, y la consola muestra “datos” y “ventas” como carpetas por conveniencia.

**Operaciones con AWS CLI**

- `aws s3 ls s3://bucket/prefix` — listar objetos
- `aws s3 cp archivo s3://bucket/key` — copiar archivo local a S3 (o viceversa)
- `aws s3 cp s3://bucket/prefix . -recursive` — descargar recursivamente

- `aws s3 sync carpeta/ s3://bucket/prefix/` — sincronizar (solo sube lo que cambió)
- `aws s3 rm s3://bucket/key` — eliminar objeto
- `aws s3 rm s3://bucket/prefix -recursive` — eliminar recursivamente

## AWS Athena

Athena es un servicio de **consultas SQL interactivas** directamente sobre archivos almacenados en S3. No requiere levantar un servidor ni cargar datos en una base de datos.

### Características:

- **Serverless:** no hay infraestructura que administrar
- **Pay-per-query:** se paga por la **cantidad de datos escaneados** por cada consulta
- **Schema-on-read:** se define un schema al consultar, no al almacenar
- Soporta formatos: CSV, JSON, Parquet, ORC, Avro

**Recordá:** en la sección de formatos de archivo vimos que **Parquet** (column-oriented) es ideal para consultas analíticas que leen pocas columnas de muchas filas. En Athena, usar Parquet en vez de CSV reduce drásticamente la cantidad de datos escaneados y por lo tanto el costo.

## AWS RDS

RDS (Relational Database Service) es un servicio **administrado** de bases de datos relacionales en la nube. Soporta PostgreSQL, MySQL, SQL Server, Oracle, entre otros.

### Diferencia con Athena:

- RDS mantiene un **servidor de base de datos corriendo** permanentemente (pay-per-hour)
- Los datos se almacenan **dentro de la base de datos**, no en S3
- Soporta **escrituras y transacciones** (OLTP)
- Athena es mejor para **análisis ad-hoc** sobre datos ya almacenados en S3

**Recordá:** la distinción OLTP vs OLAP. RDS es un servicio pensado para **OLTP** (muchas escrituras pequeñas y rápidas). Athena se alinea más con **OLAP** (consultas analíticas sobre grandes volúmenes).

## 12.2 — Organización del Master Dataset

**Recordá:** el master dataset es el **core de la arquitectura lambda**, la source of truth del sistema. Contiene datos **crudos, inmutables y perpetuos**. Ahora vemos cómo organizar su almacenamiento físico de manera eficiente.

### Particionamiento Vertical

El master dataset se organiza mediante **particionamiento vertical**: se separan los datos en **directorios** según algún criterio lógico, típicamente por **entidad** o **tipo de dato**.

**Ejemplo:** en una red social, se podrían tener carpetas separadas para:

- `/master/usuarios/` — datos de perfil de usuarios
- `/master/relaciones/` — relaciones de amistad

- `/master/interacciones/` — likes, shares, comentarios

### Ventajas:

- Las batch views que solo necesitan un tipo de dato **no escanean todo el dataset**
- Facilita la **evolución del esquema** por separado
- Mejor **paralelización**: distintas computaciones pueden procesar distintas particiones simultáneamente

En S3, esto se implementa con **prefijos de bucket**: `s3://master-dataset/usuarios/`, `s3://master-dataset/rel` etc.

## 12.3 — Servers

### Evolución de la arquitectura de servidores

La arquitectura de cómputo evolucionó en varias etapas:

1. **Mainframes (1950s–70s)**: una computadora central grande y cara. Múltiples usuarios acceden vía terminales “tontas”. Todo el procesamiento centralizado.
2. **Desktop apps (1980s–90s)**: la PC personal tiene su propio procesamiento. Los datos pueden estar en un servidor, pero la lógica corre en la máquina del usuario.
3. **HTML estático (1990s)**: páginas web servidas como archivos estáticos. El servidor solo envía archivos, no procesa lógica.
4. **Applets (Java, Flash)**: código ejecutado en el **navegador del cliente**. Problema: dependencia de plugins, seguridad, performance.
5. **HTML dinámico (2000s)**: el servidor genera HTML dinámicamente (PHP, JSP, ASP). La lógica vuelve al servidor.
6. **Arquitecturas modernas**: APIs REST/GraphQL, microservicios, SPAs con frontend desacoplado del backend.

### Computación secuencial vs paralela

**Secuencial**: un solo procesador ejecuta las instrucciones una tras otra. Simple pero limitado por la velocidad del procesador.

**Paralelo**: múltiples procesadores trabajan simultáneamente en partes distintas del problema. Requiere que el problema sea **divisible** en sub-tareas independientes.

**Ley de Amdahl**: el speedup máximo de un programa con paralelización está limitado por la **porción secuencial** del programa. Si el 10 % del programa es inherentemente secuencial, el speedup máximo teórico es 10x, sin importar cuántos procesadores se agreguen.

## 12.4 — MapReduce

**Recordá**: en la arquitectura lambda, la batch layer pre-computa **batch views** como función de todo el master dataset. MapReduce es el **paradigma de procesamiento** que permite hacer esto de forma distribuida y escalable.

### El paradigma MapReduce

MapReduce es un modelo de programación para procesar grandes volúmenes de datos en paralelo sobre un cluster de máquinas. Consta de **5 fases**:

1. **Split:** el input se divide en fragmentos (splits) que se distribuyen entre los nodos del cluster.
2. **Map:** cada nodo aplica una función `map()` a su fragmento. Emite pares (`clave`, `valor`).
3. **Shuffle & Sort:** el framework **agrupa** todos los pares por clave y los **ordena**. Pares con la misma clave llegan al mismo reducer.
4. **Reduce:** una función `reduce()` recibe todos los valores asociados a una clave y los agrega/combina en un resultado.
5. **Merge:** los resultados de todos los reducers se combinan en el output final.

### Ejemplo canónico: Word Count

Contar la frecuencia de cada palabra en un conjunto de documentos.

- **Map:** para cada palabra en el documento, emitir (`palabra`, `1`)
- **Shuffle & Sort:** agrupar por palabra: (`"hola"`, `[1, 1, 1]`)
- **Reduce:** sumar los valores: (`"hola"`, `3`)

### Hadoop MapReduce en detalle

**Recordá:** Hadoop tiene tres componentes: HDFS (almacenamiento distribuido), MapReduce (procesamiento) y YARN (gestión de recursos). Ahora profundizamos en cómo MapReduce ejecuta un job sobre HDFS.

#### Arquitectura HDFS:

- **NameNode:** nodo maestro que mantiene los **metadatos** del filesystem (qué archivo está en qué bloque, en qué nodo). No almacena datos.
- **DataNodes:** nodos que almacenan los **bloques de datos**. Cada bloque se replica (por defecto 3 veces) para tolerancia a fallos.

#### Pasos de ejecución de un job MapReduce en Hadoop:

1. **Splitting + Mapping:** los datos de HDFS se dividen en splits. Cada mapper procesa un split y escribe los resultados intermedios al **disco local** (no a HDFS).
2. **Partitioning:** los resultados del map se particionan por clave para determinar a qué reducer irá cada par.
3. **Shuffling + Sorting:** los datos se transfieren por red desde los mappers a los reducers, ordenados por clave.
4. **Reducing:** cada reducer procesa todas las claves que le corresponden y escribe el resultado final a **HDFS**.

**Data locality (localidad de datos):** Hadoop intenta ejecutar el mapper en el **mismo nodo** donde están almacenados los datos. Mover el procesamiento hacia los datos es mucho más eficiente que mover los datos hacia el procesamiento.

**YARN:** se encarga de la **gestión de recursos** del cluster. Tiene un **ResourceManager** (global) que asigna recursos, y **NodeManagers** (por nodo) que monitorean el uso de CPU, memoria y disco. El **ApplicationMaster** coordina la ejecución de cada job.

## 12.5 — Spark

## ¿Qué es Spark?

Apache Spark es un motor de procesamiento distribuido que, a diferencia de Hadoop MapReduce, opera **en memoria** (in-memory). Esto lo hace significativamente más rápido para workloads iterativos y consultas interactivas.

**Recordá:** Hadoop MapReduce escribe resultados intermedios al **disco local** entre cada fase Map y Reduce. Spark mantiene los datos en **memoria RAM** entre operaciones, evitando la latencia de I/O a disco. Esto es especialmente ventajoso en algoritmos iterativos (como ML) donde el mismo dataset se recorre múltiples veces.

## RDD (Resilient Distributed Dataset)

El RDD es la abstracción fundamental de Spark. Es una **colección distribuida e inmutable** de elementos que puede ser procesada en paralelo.

### Propiedades:

- **Resilient:** tolerante a fallos. Si se pierde una partición, se puede recalcular a partir de su **lineage** (la secuencia de transformaciones que la creó).
- **Distributed:** los datos se distribuyen entre los nodos del cluster.
- **Immutable:** una vez creado, no se modifica. Las transformaciones generan **nuevos** RDDs.

### Dos tipos de operaciones sobre RDDs:

- **Transformations:** operaciones **lazy** que definen un nuevo RDD sin ejecutarlo inmediatamente. Ejemplos: `map`, `filter`, `flatMap`, `reduceByKey`, `groupByKey`.
- **Actions:** operaciones que **disparan la ejecución** del plan y devuelven un resultado. Ejemplos: `collect`, `count`, `saveAsTextFile`, `reduce`.

Spark construye un **DAG (Directed Acyclic Graph)** de transformaciones y solo ejecuta cuando se invoca una action (**lazy evaluation**).

## DataFrames

Los DataFrames son una abstracción de más alto nivel sobre los RDDs. Representan datos como una **tabla con columnas nombradas y tipadas**, similar a una tabla SQL o un DataFrame de Pandas.

### Ventajas sobre RDDs:

- **Optimización automática:** Spark SQL usa un **query optimizer** (Catalyst) que optimiza el plan de ejecución.
- **API más simple:** operaciones tipo SQL (`select`, `filter`, `groupBy`, `join`).
- **Interoperabilidad:** se pueden consultar con SQL directamente.

## Data Skew (sesgo de datos)

El **data skew** es un problema crítico en procesamiento distribuido. Ocurre cuando la distribución de claves es **desigual**: algunas claves tienen muchísimos más valores que otras.

**Consecuencia:** el reducer que maneja la clave “popular” recibe mucho más trabajo que los demás. Todo el job queda esperando a ese único reducer lento — se pierde el beneficio de la paralelización.

### Mitigaciones:

- **Salting:** agregar un prefijo aleatorio a las claves para distribuir mejor la carga, y luego hacer una segunda agregación.



- **Repartitioning:** redistribuir los datos explícitamente.
- **Broadcast join:** si una tabla es pequeña, enviarla completa a todos los nodos en vez de hacer shuffle.

## MapReduce vs Spark

| Aspecto                   | Hadoop MapReduce                   | Spark                                         |
|---------------------------|------------------------------------|-----------------------------------------------|
| Almacenamiento intermedio | Disco local                        | Memoria RAM                                   |
| Latencia                  | Alta (I/O a disco)                 | Baja (in-memory)                              |
| Iteraciones               | Lento (lee/escribe disco cada vez) | Rápido (datos en memoria)                     |
| Tolerancia a fallos       | Re-lee desde disco                 | Recalcula desde lineage                       |
| Robustez                  | Muy estable para jobs enormes      | Puede fallar si los datos no caben en memoria |
| Costo                     | Más barato (disco es barato)       | Más caro (RAM es cara)                        |

## 12.6 — Adecuación a Big Data

**Recordá:** la batch layer genera **batch views** como función del master dataset: `batch view = f(all data)`. Esta sección muestra ejemplos concretos de batch views que se pueden generar con MapReduce o Spark.

### Ejemplo 1: Clasificador Bayesiano de Spam

Un clasificador Naive Bayes para spam se puede entrenar con MapReduce. El modelo necesita calcular probabilidades condicionales:  $P(\text{palabra}|\text{clase})$  para  $\text{clase} \in \{\text{spam}, \text{ham}\}$ .

**Mapper:** para cada email del dataset de entrenamiento, emite:

- `(TOTAL_SPAM, 1)` o `(TOTAL_HAM, 1)` — conteo total por clase
- `(COUNT_SPAM_palabra, 1)` o `(COUNT_HAM_palabra, 1)` — conteo de cada palabra por clase
- `(TOTAL_WORD_SPAM, 1)` o `(TOTAL_WORD_HAM, 1)` — total de palabras por clase

**Reducer:** suma todos los valores para cada clave. El resultado es una tabla de frecuencias que permite calcular las probabilidades del modelo.

**Recordá:** este clasificador es un ejemplo de **batch view** generada por la batch layer. El modelo entrenado se almacena en la serving layer y se usa para clasificar nuevos emails en tiempo real.

### Ejemplo 2: Índice Invertido

Un **índice invertido** mapea cada palabra a la lista de documentos donde aparece (y opcionalmente la posición). Es la base de los motores de búsqueda.

**Con MapReduce:**

- **Map:** para cada palabra en un documento, emitir `(palabra, (doc_id, posición))`
- **Reduce:** para cada palabra, recolectar todas las tuplas `(doc_id, posición)` en una lista

**Con Spark:**

```
rdd.flatMap(lambda doc: [(word, (doc.id, pos))
                          for pos, word in enumerate(doc.words)])
      .groupByKey()
      .mapValues(list)
```

### Ejemplo 3: Filtrado Colaborativo (Sistemas de Recomendación)

El filtrado colaborativo busca recomendar ítems basándose en los **patrones de preferencia de usuarios similares**.

**Modelo:** se tiene una matriz  $R$  de  $n$  usuarios  $\times$   $m$  ítems, donde  $R_{ij}$  es el rating que el usuario  $i$  dio al ítem  $j$ . La mayoría de las entradas están vacías (la matriz es **sparse**).

**Factorización matricial:** se buscan dos matrices  $U$  ( $n \times k$ ) y  $V$  ( $k \times m$ ) tales que  $R \approx U \cdot V$ . Cada usuario queda representado por un vector de  $k$  **factores latentes**, y cada ítem igual. El producto punto entre el vector de un usuario y el de un ítem predice el rating.

**Con MapReduce/Spark:** el entrenamiento es iterativo (ej: ALS — Alternating Least Squares). En cada iteración:

- Se fija  $V$  y se optimiza  $U$  (paralelizable por usuario)
- Se fija  $U$  y se optimiza  $V$  (paralelizable por ítem)

**Recordá:** los algoritmos iterativos como ALS son donde Spark brilla frente a Hadoop MapReduce. Hadoop escribiría a disco después de cada iteración; Spark mantiene las matrices en memoria entre iteraciones, reduciendo drásticamente la latencia.